Código de Três Endereços: Instruções, Conversões de Tipo e Exemplos

1. Introdução

O **Código de Três Endereços** (*Three-Address Code – TAC*) é uma representação intermediária amplamente utilizada em compiladores. Ele é gerado a partir da análise sintática e semântica do programa-fonte e representa as operações de maneira mais simples e próxima de uma linguagem de baixo nível.

O objetivo principal do código de três endereços é decompor expressões complexas em uma sequência de instruções simples. Normalmente, cada instrução contém, no máximo, três referências a endereços, que podem representar variáveis, constantes, temporários ou rótulos.

Por exemplo, a expressão:

```
a = b + c * d;
```

pode ser convertida para:

```
t1 = c * d

t2 = b + t1

a = t2
```

As variáveis t1 e t2 são chamadas de temporários e são utilizadas para armazenar resultados intermediários.

2. Elementos do Código de Três Endereços

Um código de três endereços pode utilizar os seguintes elementos:

- Variáveis: identificadores declarados no programa.
- Constantes: valores literais.
- Temporários: variáveis criadas pelo compilador para armazenar resultados intermediários.
- Operadores: operadores aritméticos, relacionais, lógicos e outros.
- Rótulos: identificadores utilizados como destinos de desvios.
- Instruções de salto: utilizadas para implementar estruturas de controle.
- Conversões de tipo: utilizadas quando uma operação exige compatibilidade entre tipos diferentes.

```
t1 = b * c

t2 = a + t1

x = t2
```

3. Instruções de Atribuição

A forma mais simples de uma instrução é uma atribuição.

3.1. Atribuição simples

Forma geral:

```
x = y
```

Exemplos:

```
a = b
```

```
a = 10
```

4. Operações Binárias

As operações binárias possuem dois operandos e um operador.

Forma geral:

```
x = y op z
```

Operadores aritméticos comuns:

```
+
-
*
/
%
```

Exemplo:

```
a = b + c * d;
```

```
t1 = c * d
```

```
t2 = b + t1
```

```
a = t2
```

5. Operações Unárias

As operações unárias utilizam apenas um operando.

Forma geral:

```
x = op y
```

5.1. Negação aritmética

```
a = -b;
```

```
t1 = -b
```

```
a = t1
```

5.2. Negação lógica

```
a = !b;
```

```
t1 = !b
```

```
a = t1
```

6. Operações Relacionais

Os operadores relacionais são utilizados para comparar valores. Os principais operadores são:

```
<
>
<=
>=
==
!=
```

Exemplo:

```
x = a < b;
```

```
t1 = a < b
```

```
x = t1
```

Também é possível utilizar diretamente a comparação em uma instrução condicional:

```
if a < b goto L1
```

7. Operações Lógicas

As operações lógicas mais comuns são AND, OR e NOT, ou, dependendo da notação, &&, || e !.

```
x = a && b;
```

```
t1 = a && b
```

```
x = t1
```

```
x = (a > b) || (c > d);
```

```
t1 = a > b
```

```
t2 = c > d
```

```
t3 = t1 || t2
```

```
x = t3
```

8. Desvio Incondicional

O desvio incondicional transfere o fluxo de execução para um determinado rótulo.

```
goto L1
```

```
...
```

```
L1:
```

9. Desvio Condicional

O desvio condicional permite alterar o fluxo de execução dependendo de uma condição. Formas comuns:

```
if x goto L


if x < y goto L


if (a < b)
    x = 1;


if a < b goto L1
    goto L2
L1:
    x = 1
L2:
```

10. Rótulos

Os rótulos identificam posições específicas no código intermediário e são utilizados como destino de instruções de desvio.

```
L1:
    t1 = a + b


goto L2

...
L2:
    x = t1
```

Os rótulos são frequentemente utilizados para representar comandos if, if-else, while, for, do-while, break e continue.

11. Conversões de Tipo

As conversões de tipo são importantes na geração de código intermediário quando uma expressão envolve valores de tipos diferentes.

```
int i;
float f;

f = i + 2.5;

t1 = inttofloat(i)
t2 = t1 + 2.5
f = t2
```

12. Conversão Implícita de Tipo

A conversão implícita ocorre quando o compilador realiza automaticamente a conversão para compatibilizar os tipos.

```
int i = 10;
float f;

f = i + 2.5;
```

```
i = 10
t1 = inttofloat(i)
t2 = t1 + 2.5
f = t2
```

13. Principais Conversões de Tipo

Uma implementação de código de três endereços pode utilizar instruções específicas para conversão. Alguns exemplos são:

```
inttofloat(x)
inttodouble(x)
inttochar(x)

floattoint(x)
floattodouble(x)

doubletoint(x)
doubletofloat(x)

chartoint(x)
chartofloat(x)
```

Exemplo com int, float e double

```
int i;
float f;
double d;

d = i + f;
```

```
t1 = inttofloat(i)
t2 = t1 + f
t3 = floattodouble(t2)
d = t3
```

Dependendo das regras da linguagem, o compilador pode converter ambos os operandos para double antes da operação:

```
t1 = inttodouble(i)
t2 = floattodouble(f)
t3 = t1 + t2
d = t3
```

14. Acesso a Vetores

```
x = a[i];

t1 = i * 4
t2 = a + t1
x = *t2
```

15. Atribuição em Vetores

```
a[i] = x;

t1 = i * 4
t2 = a + t1
*t2 = x
```

16. Acesso Indireto por Ponteiros

```
x = *p;
*p = x;

x = *p
*p = x
```

17. Operações de Endereço

Obtenção de endereço:

```
p = &x;

p = &x
```

Acesso indireto:

```
y = *p;

y = *p
```

18. Chamada de Funções

O código de três endereços pode representar chamadas de funções utilizando instruções como:

```
param x
call func, n


z = soma(a, b);

param a
param b
```

```
t1 = call soma, 2
z = t1
```

19. Retorno de Funções

```
int soma(int a, int b) {
    return a + b;
}

soma:
t1 = a + b
return t1
```

Uma função sem valor de retorno pode ser representada por:

```
return
```

20. Instruções para Arrays, Ponteiros e Funções

```
x = y
x = y op z
x = op y

x = &y
x = *y
*x = y

x = y[i]
x[i] = y

param x
x = call f, n
return x
return
```

Resumo das Principais Instruções

Instrução	Descrição	Exemplo
x = y	Atribuição simples	a = b
x = constante	Atribuição de constante	a = 10
x = y op z	Operação binária	t1 = a + b
x = op y	Operação unária	t1 = -a
x = (tipo) y	Conversão de tipo	t1 = (float) a
x = inttofloat(y)	Conversão de int para float	t1 = inttofloat(a)
x = floattoint(y)	Conversão de float para int	t1 = floattoint(a)
goto L	Desvio incondicional	goto L1
if x goto L	Desvio condicional	if x goto L1
if x relop y goto L	Comparação com desvio	if a < b goto L1
L:	Rótulo	L1:
x = y[i]	Leitura de vetor	t1 = a[i]

x[i] = y	Escrita em vetor	a[i] = x
x = &y	Obtenção de endereço	p = &x
x = *y	Acesso indireto	x = *p
*x = y	Escrita indireta	*p = x
param x	Passagem de parâmetro	param a
x = call f, n	Chamada de função	t1 = call soma, 2
return x	Retorno com valor	return t1
return	Retorno sem valor	return

Exemplo Completo

```
int a;
float b;
float c;

a = 10;
b = 2.5;

if (a > 5) {
    c = a + b;
} else {
    c = b * 2;
}
```

Uma possível representação em código de três endereços é:

```
a = 10
b = 2.5

ifFalse a > 5 goto L1

t1 = inttofloat(a)
t2 = t1 + b
c = t2

goto L2

L1:
t3 = b * 2
c = t3

L2:
```

Nesse exemplo podem ser observados:

- atribuições;
- constantes;
- operação relacional;
- desvio condicional;
- rótulos;
- conversão de int para float;
- operação aritmética;
- desvio incondicional.